SECTION 1: INTRODUCTION TO MICROPROCESSORS

Q1. What are the differences between microprocessors and microcontrollers?
A **microprocessor (μP)** is the central processing unit (CPU) of a computer. It performs computation, logic, and control but lacks memory or I/O peripherals. It needs **external RAM, ROM, and I/O ports** to function. Thus, it is ideal for general-purpose computing systems like PCs.

A **microcontroller (μC)** is a compact, self-contained chip that integrates **CPU + Memory (RAM, ROM/Flash) + I/O ports + Timers + ADC** on a single chip. It is designed for specific control-oriented applications like home appliances, IoT devices, and automobiles.

Feature	Microprocessor	Microcontroller
Components	CPU only	CPU + RAM + ROM + I/O + Timers on one chip
System Cost	Higher (external components)	Lower (integrated design)
Power Consumption	Higher	Lower
Speed	Very high, optimized for processing	Moderate, optimized for control
Example	Intel 8086, Pentium	8051, ATmega, PIC

Q2. Explain the major components of a CPU and their functions.
The **CPU** is the brain of a computer, controlling all processing activities. It contains: 1. **Arithmetic Logic Unit (ALU):** Performs arithmetic operations (addition, subtraction, multiplication, division) and logical operations (AND, OR, NOT, XOR). 2. **Control Unit (CU):** Directs the sequence of operations. It fetches, decodes, and executes instructions. It generates control signals for the buses and memory. 3. **Registers:** High-speed storage inside the CPU. Includes general-purpose registers (AX, BX, CX, DX), pointer/index registers (SP, BP, SI, DI), and special registers (PC/IP, MAR, MDR, IR). Together they perform the **Fetch–Decode–Execute cycle**.

Q3. For each device, choose between microcontroller or microprocessor. | Device | Component Type | Reason | |---------|---------------|--------| | Keyboard | Microcontroller | Performs simple key scanning & sends signals | | Mouse | Microcontroller | Manages motion sensors and buttons | | Headphone | Microcontroller | Controls Bluetooth and volume | | Computer | Microprocessor | Requires multitasking and OS support | | TV Remote | Microcontroller | Handles limited control functions | | Smart TV | Microprocessor | Runs apps, OS, and heavy processing | | Regular Fridge | Microcontroller | Manages

temperature sensors and compressor | | Mobile Phone | Microprocessor (SoC) | High-speed CPU, GPU, multitasking |

Q4. Smart home system design — µP or µC?
A **microcontroller** is more suitable because it offers embedded control with low power, integrated sensors, and I/O interfaces. It can handle local control tasks (e.g., lighting, door locks) efficiently. Only a central hub managing data or AI analytics may use a microprocessor.

Q5. Processor performance factors:
Performance is affected by multiple hardware and architectural parameters: - **Clock frequency (Hz):** Higher clock = more instructions per second. - **Instruction set efficiency:** Simpler instruction sets execute faster. - **Bus width:** Wider data/address buses transfer more data at once. - **Cache and pipeline:** Reduce memory access latency. - **Memory latency:** Slower memory reduces overall performance.

Thus, an 8-bit processor is not always faster than a 4-bit one — architecture and design play a crucial role.

Q6. Why microcontroller for washing machine control?
Microcontrollers combine ROM, RAM, timers, I/O ports, and CPU in a single low-cost chip. They handle repeated control operations such as motor speed, water level, and time delay efficiently. Their low power and interrupt features make them ideal for real-time embedded control.

Q7. RISC or CISC for high-performance computing?
RISC (Reduced Instruction Set Computer) is preferred for high-performance computing due to: - Fewer, simpler instructions that execute in one cycle. - Uniform instruction length → easier pipelining. - Higher instruction throughput. In contrast, **CISC** (Complex Instruction Set Computer) supports more complex instructions but executes slower per instruction.

Q8. Choosing µP or µC for a gaming device.
Use a **microprocessor**, since gaming devices require high-speed computation, graphics rendering, and multitasking. Microcontrollers are designed for low-power, single-task control operations and cannot handle large graphics or OS operations.

Q9. Simple arithmetic program:

```
MOV AL, 7      ; Store value 7 in AL
ADD AL, 3      ; AL = 7 + 3 = 10
SUB AL, 5      ; AL = 10 - 5 = 5
```

Final Result → AL = 5.

SECTION 2: OVERVIEW OF MICROCOMPUTER STRUCTURE & OPERATION

Q1. Block diagram of a Microcomputer:
Consists of CPU, memory, I/O units interconnected by three buses (address, data, control). CPU includes ALU, CU, and registers.

Q2. Differentiate RAM and ROM:
RAM is temporary (volatile), stores data and instructions during execution. ROM is permanent (non-volatile), stores firmware and bootstrap code.

Q3. System buses:
- **Address Bus:** Unidirectional, selects memory locations. Width determines maximum addressable memory. - **Data Bus:** Bidirectional, transfers data between CPU and memory/I/O. - **Control Bus:** Carries signals like RD, WR, MEMR, IOR for synchronization.

Q4. Which CPU part handles calculation?
ALU performs arithmetic and logical operations.

Q5. Primary role of Address Bus:
To carry the address from CPU to memory or I/O device.

Q6. Temporary data storage component:
Registers store frequently used values or addresses for faster processing.

Q7. Functions of CPU:
1. **Fetch**: Retrieve instruction from memory using PC. 2. **Decode**: Control unit interprets opcode. 3. **Execute**: ALU or I/O performs the action. 4. **Store**: Write result back to register/memory.

Q8. Why address bus unidirectional & data bus bidirectional?
Address bus carries address only from CPU → memory/I/O, whereas data bus carries values both ways.

Q9. Which memory for startup instructions?
ROM, because it retains data after power loss.

Q10. Role of Program Counter:
Holds address of next instruction to be executed.

Q11. MAR vs MDR:
MAR stores memory address; MDR stores data being transferred.

Q12. 16-bit address bus → memory size:
2^16 = 65,536 bytes = 64 KB.

Q13. Slow execution cause:
Differences in instruction complexity — long instructions spend more time in the **Decode** or **Execute** phase.

Q14. Data-intensive applications and data bus size:
Increasing data bus width improves throughput (more bits transferred per cycle). But improvement depends on whether memory and I/O can match the bus speed.

Q15. Control bus role:
Manages communication control — read/write, interrupts, acknowledgment, and timing. It ensures the CPU and peripherals synchronize correctly.

SECTION 3: 8086 MEMORY ADDRESS SPACE PARTITION

Q1. Deduce address bus size if total memory = 4 MB.
4 MB = 4 × 1024 × 1024 = 4,194,304 bytes = 2^22 → **22 address lines.**

Q2. Total memory size for 21-bit address bus:
2^21 = 2,097,152 bytes = **2 MB**.

Q3. Logical vs Physical Address:
Logical = Segment:Offset (used by programmer). Physical = (Segment × 10h) + Offset (used by hardware).

Q4. Segment Registers:
CS (Code), DS (Data), SS (Stack), and ES (Extra) registers hold the starting base addresses for different 64 KB memory segments.

Q5. Overlapping vs Non-overlapping Segments:
Overlapping segments share part of memory (e.g., 1000h and 1001h), while non-overlapping segments differ by ≥ 1000h.

Q6. Advantages of segmentation:
- Enables 1 MB addressing with 16-bit registers. - Logical separation for code, data, and stack. - Simplifies relocation and modular programming.

Q7. Segment registers vs general registers:
Segment registers define memory blocks; general-purpose registers store data and operands. Segment registers cannot perform arithmetic.

Q8. Segment=A4FBh, Offset=4872h → Physical Address:
Formula: Physical = (Segment × 10h) + Offset
→ A4FB × 10h = A4FB0h
→ A4FB0h + 4872h = **A9822h**.

Q9. Segment 20000h and 20100h Overlap Explanation:
Each segment = 64 KB = 10000h bytes. - Segment 1 range = 20000h → 2FFFFh - Segment 2 range = 20100h → 300FFh Because the second starts before the first ends, their ranges **overlap from 20100h to 2FFFFh.** ✅
Answer: Yes, they overlap.

Q10. Segment=1111h, Offset=1332h → Physical:
Physical = (1111 × 10h) + 1332h = 11110h + 1332h = **12442h**.

Q11. Physical=33330h, Offset=0020h → Segment:
Segment × 10h = 33330h − 0020h = 33310h → Segment = **3331h**.

Q12. Convert physical to segment:offset:
A9822h → A4FB:4872
38441h → 3840:0441

Q13. How many 64KB segments in 1MB?
1MB = 1024KB → 1024/64 = **16 segments (non-overlapping)**.

Q14. CS=4321h, IP=1234h → Physical:
(4321h × 10h) + 1234h = 43210h + 1234h = **44444h.**

Q15. Fifth largest and smallest segment-offset pairs (detailed):
Given physical = 32556h. Formula:

$$Physical = (Segment \times 10h) + Offset$$

Step 1: Valid offset range

Offset ≤ FFFFh (max 16-bit).
So find all segment values where the offset remains within that limit.

Step 2: Find maximum and minimum possible segment values

Maximum segment (offset = 0):

$$32556h/10h = 3255h$$

→ Segment(max) = 3255h.

Minimum segment (offset = FFFFh):

$$32556h - (Segment \times 10h) \leq FFFFh$$

$$(Segment \times 10h) \geq 2257h$$

→ Segment(min) ≈ 2257h.

Step 3: Listing possible segments

2257h, 2258h, 2259h, ..., 3255h.
5th smallest = 225Bh
5th largest = 3251h

Step 4: Calculate offsets

For 225Bh: Offset = 32556h − 225B0h = 0xFFF6h
For 3251h: Offset = 32556h − 32510h = 0x0046h

✅**5th smallest → 225B:FFF6**
✅**5th largest → 3251:0046**

END OF SOLUTION SET (Elaborated Version)